

Constraining Concepts Overload Sets

ISO/IEC JTC1 SC22 WG21 P0782R1

ADAM David Alan Martin (adam@recursive.engineer)
Erich Keane (erich.keane@intel.com)

Abstract

The central purpose of Concepts is to simplify generic programming such that it is approachable to the non-expert developer. In general it makes great strides towards this end particularly in the capacity of invoking a generic function; however, the Concepts design does not deliver on this promise in the implementation of a generic function. This is because the feature does not constrain the overload set of a template-concept function itself. This is contrary to the expectations of non-experts, because to them Concepts should strongly resemble the callable properties of an interface. This mental model drives their expectations to believe that Concepts offer a mechanism to limit the set of operations which would be visible from within their constrained function to those which are specified by concept used by the constrained function.

The fact that this is not the case in constrained functions can lead to surprising violations of the author's expectations thereof. Unfortunately, this oversight cannot be corrected later. To correct this later would entail silent behavioral changes to existing code after the release of Concepts in a standard. In other words, this is our only chance to get this right.

Simple Motivating Example

```
// Assume a concept, `IntegerSerializer` which requires a member function called `serialize` which  
// takes an `int` and returns a `std::string` with the representation of that integer, in the format  
// appropriate to the implementation.
```

```
template< typename Instance >  
concept IntegerSerializer= requires( Instance instance, int value )  
{  
    { std::as_const( instance ).serialize( value ) } -> void;  
}
```

```
class SimpleSerializer  
{  
    std::string serialize( int value ) const;  
};
```

```
struct PrecisionSerializer  
{  
    std::string serialize( double preciseValue ) const;  
};
```

```
struct LoanInterestSerializer  
{  
    std::string serialize( int interestBasisPoints ) const;  
    std::string serialize( double interestRate ) const;  
};
```

```
// This is a terse-syntax style template function. Whether the author knew he or  
// she was writing a template is not necessarily clear.  
std::string formatLogarithmicValue( IntegerSerializer &serializer, int integerValue )  
{  
    return serializer.serialize( std::log( integerValue ) );  
}
```

The issue is in what kind of object we pass to `formatLogarithmicValue`. If we pass an `SimpleSerializer`, then no surprises happen -- the double is implicitly converted to int. If we pass `PrecisionSerializer`, then the definition of the concept will pass and an implicit conversion to int will happen, which is potentially surprising; however, many programmers are reasonably comfortable with the idea of fundamental type conversions. The most surprising case is that of `LoanInterestSerializer`. `LoanInterestSerializer` provides a double and an int overload. Although the concept requested function with a signature that accepts an int, the overload which accepts double in its signature will be called.

From the perspective of the compiler, the way it will actually compile the function `formatLogarithmicValue` is as if it were written:

```
template< typename IntegerSerializer >
std::string
formatLogarithmicValue( IntegerSerializer &serializer, int integerValue )
{
    return serializer.serialize( std::log( integerValue ) );
}
```

At this point, the invocation of `IntegerSerializer::serializer` will be whatever best matches `decltype( std::log( integerValue ) )`, which is the overload with double as its parameter. This is likely surprising behavior to the author of `formatLogarithmicValue`, as well as the caller of `formatLogarithmicValue`. Both of these authors would expect that the constraints described by the concept would be obeyed, yet paradoxically the overload which was not the best match for the constraint was actually the overload that was actually invoked in the body of the "constrained" function!

The only way for an author of such a constrained function to avoid this, at present, is to rewrite `formatLogarithmicValue` in such a way as to prevent the incorrect lookup. Unfortunately, this requires a level of C++ expertise regarding name lookup and overload resolution which is at odds with the level of expertise expected of the audience of Concepts, viz. the non-expert programmer. Such a rewrite might appear thus:

```
template< typename IS >
std::string
formatLogarithmicValue( IS &serializer, int integerValue )
    requires( IntegerSerializer< IS > )
{
    return std::as_const( serializer ).serialize(
        static_cast< int >( std::log( static_cast< double >( integerValue ) ) ) );
}
```

This does not appear to be code that would be expected of the audience targetted by Concepts. Additionally, although one of the authors of this paper is author of `std::as_const`, this is not the purpose nor audience he had in mind when he proposed it.

The primary purpose of the static casts and as-consts in this rewrite are actually dedicated to the selection of the correct overload, not to any specific need to have a value in the form of any specific type.

An Example at Scale

```
namespace ConceptLibrary
{
    template< typename Instance >
    concept Stringable= requires( Instance instance, int value )
    {
        { std::as_const( instance ).toString() } -> std::string;
    }
}

namespace PayrollLibrary
{
    class Employee
    {
        private:
```

```

        std::string name;

public:
    Employee( std::string initialName );

    // This satisfies the Stringable concept, and the author of this type knows that.
    std::string toString() const;

    friend bool operator == ( const Employee &lhs, const Employee &rhs );

    // The following functions are friend, to indicate that ADL is intended.

    // Terminate the specified employee's employment
    friend void fire( const Employee &emp );

    // Initiate the specified employee's employment
    friend void hire( const Employee &emp );

    // Returns true if the specified employee is employed and false otherwise.
    friend bool worksHere( const Employee &emp );
};
}

namespace AlgorithmLibrary
{
    // This "fires" off a stringable object to be processed. It is a terse-syntax style
    // template function. Whether the author knew he or she was writing a template is
    // not necessarily clear. Although it is likely that the author knew the function
    // was a generic function.
    void
    fire( const ConceptLibrary::Stringable &s )
    {
        std::cout << "I am interested in " << s.toString() << std::endl;
    }

    // This too is a terse-syntax style template function. Whether the author knew he or
    // she was writing a template is not necessarily clear. Although it is likely that the
    // author knew the function was a generic function. The purpose of this function is
    // to call the above helper function in a loop.
    void
    printAll( const std::vector< ConceptLibrary::Stringable > &v )
    {
        for( auto &&s: v ) fire( s );
    }
}

namespace UserProgram
{
    void code()
    {
        std::vector< PayrollLibrary::Employee > team;
        team.emplace_back( "John Doe" );
        AlgorithmLibrary::printAll( team );
    }
}

```

In this example the intent of the three separate authors is apparent. The author of the `PayrollLibrary` simply wished to afford his users the ability to represent employees at a company. The author of the `AlgorithmLibrary` wished to afford his users the ability to print printable things and needed to write an internal helper method to better organize his code. The author of `UserProgram` naively wished to combine these reusable components for a simple task. However, a subtle behavior of name lookup in function templates resulted in the termination of employees, rather than the intended call to an implementation detail. This happened because the name of the implementation detail happened to collide with the name of some irrelevant API in the `Employee` object being processed. Recall that the author of `AlgorithmLibrary` is likely unaware of the fact that types may exist which are `Stringable` and yet interfere with the name he chose for his internal implementation detail.

The authors of this paper recognize the importance of respecting the original intent of the programmers of these components without burying them in the details of defensive template writing. These kinds of examples will come up frequently and perniciously in codebases which import third party libraries and work in multiple groups each with different naming conventions. Even without variance in naming conventions, names that have multiple meanings to multiple people are likely to be used across disparate parts of a codebase, and thus they are more likely to exhibit this pathological behavior.

Why it is Important to Address this Now

Should the terse syntax be accepted into the current standard, without addressing this issue, then future attempts to repair this oversight in the language specification, leave us with one of two incredibly unpalatable alternatives and one unsatisfying one:

1. Make constraint violations an error, thus requiring extreme verbosity.
2. Silently change the meaning of existing code, with ODR implications, because these constrained functions are templates.
3. Introduce a new syntax for indicating that a function definition should be processed in a manner which is more consistent with average programmer expectations;

In the first case, vast amounts of code will fail to compile in noisy ways. After that point, the user code would need to be rewritten, in a manner similar to the necessary rewrites as described above.

In the second case, massive fallout from ODR, silent subtle semantic changes, and other unforeseen dangers lie in wait.

In the third case, the benefits of the "natural" syntax are lost, as the best syntax for beginners is no longer the natural syntax! This obviously defeats the intended purpose of Concepts with a natural syntax.

Some Design Philosophy

There are other cases where current Concepts can fail to prevent an incorrect selection of operation. This fails to deliver upon a big part of the expected benefits of this language feature. The comparison has been drawn between C++ Virtual Functions and Concepts. As Concepts are being presented to bring generic programming to the masses, it is vital that 3 core safety requirements be considered. These requirements are similar to aspects of Object Oriented Programming.

1. An object passed to a function must meet the qualifications that a concept describes. This is analogous to how a function taking a pointer or reference to a class has the parameter checked for substitutability. The current Concepts proposal provides this extremely well.
2. An object written to be used as a model of a concept should have its definition checked for completeness by the compiler. This is analogous to how a class is checked for abstractness vs concreteness. The current Concepts proposal lacks this. However, this is approximated very well by the concept checking machinery. This guarantees that every class which is matched to concept provides a definition for every required operation under that concept, thus satisfying the requirements of the concept.
3. A constrained function is only capable of calling the functions on its parameters that are described by its constraining Concepts. This is analogous to how a function taking a pointer to base is only allowed to call members of the base -- new APIs added in any derived class are not considered to be better matches, ever. The current Concepts proposal lacks anything resembling this, and this oversight has yet to be addressed. It is this deficiency which our paper seeks to remedy.

We propose that the Concepts feature is incomplete without constrained overload set for usage, thus satisfying the third requirement of any interface-like abstraction. It is vital that we explore this issue.

We recognize that some complexity in the space of constrained generics will always be present, but we feel that it is best to offload this complexity to the author of a concept rather than to the implementor of a constrained function. This is because we believe that fewer concept authors will exist than concept "users". Additionally, the level of expertise of a concept author is inherently higher than the intended audience of constrained functions. In the worst case scenario, a

naive definition of a concept will merely result in a few missed opportunities for more suitable overloads to handle move semantics, avoid conversions, and other shenanigans

What This Paper is NOT Proposing

1. Any requirement on optimizers to make any aspect of the code generated by this solution more efficient
2. Definition Checking
3. C++0x Concept Checking
4. C++0x Concept Maps
5. The generation of "invisible" proxy types
6. The generation of "invisible" inline proxy functions
7. Dynamic dispatch
8. Function call tables
9. Implicit generation of adaptors
10. Any form of extra code generation
11. Any form of extra type generation

Our Proposed Solution

We propose a moderate alteration of the overload resolution rules and name lookup rules that statically filters out some overloads based upon whether those functions are used to satisfy the concept's requirements. This constrained overload set hides some functions from visibility in the definition of constrained functions. No change should be necessary to the rules of name lookup itself; however, our new overload resolution rules will affect the results overload resolution on unqualified name lookup in constrained functions-- this is by design. We also suggest that it might be desirable to borrow a keyword (we suggest `explicit`) to indicate that this amended lookup rule should be followed. The need for this keyword to enable these lookup rules will be discussed further in the "Design Considerations" section of this paper.

Specifically, our design is to change overload resolution to be the following (taken from cpreference.com's description of the overload resolution rules):

Current overload resolution	Desired overload resolution
Given the set of candidate functions, constructed as described above, the next step of overload resolution is examining arguments and parameters to reduce the set to the set of viable functions To be included in the set of viable functions, the candidate function must satisfy the following: 1. If there are M arguments, the candidate function that has exactly M parameters is viable 2. If the candidate function has less than M parameters, but has an ellipsis parameter, it is viable. 3. If the candidate function has more than M parameters and the M+1'st parameter and all parameters that follow must have default arguments, it is viable. For the rest of overload resolution, the parameter list is truncated at M. 4. If the function has an associated constraint, it must be satisfied (since C++20) 5. For every argument there must be at least one implicit conversion sequence that converts it to the corresponding parameter.	Given the set of candidate functions, constructed as described above, the next step of overload resolution is examining arguments and parameters to reduce the set to the set of viable functions To be included in the set of viable functions, the candidate function must satisfy the following: 1. If there are M arguments, the candidate function that has exactly M parameters is viable 2. If the candidate function has less than M parameters, but has an ellipsis parameter, it is viable. 3. If the candidate function has more than M parameters and the M+1'st parameter and all parameters that follow must have default arguments, it is viable. For the rest of overload resolution, the parameter list is truncated at M. 4. If the function has an associated constraint, it must be satisfied (since C++20) 5. If at least one of the arguments to the function is constrained and that function was found by

6. If any parameter has reference type, reference binding is accounted for at this step: if an rvalue argument corresponds to non-const lvalue reference parameter or an lvalue argument corresponds to rvalue reference parameter, the function is not viable.

unqualified name lookup and the lookup found a name that is otherwise not visible at the calling location, then the function is only viable if that function was necessary to satisfy the argument's concept constraint. (this paper)

6. For every argument there must be at least one implicit conversion sequence that converts it to the corresponding parameter.
7. If any parameter has reference type, reference binding is accounted for at this step: if an rvalue argument corresponds to non-const lvalue reference parameter or an lvalue argument corresponds to rvalue reference parameter, the function is not viable.

Objections, Questions, and Concerns

Q: Isn't this just C++0x Concepts with definition checking all over again?

A: No. C++0x Concepts used mechanisms and techniques which are drastically different to the solution we have proposed. In the original C++0x Concepts, the compiler was required to create invisible wrapping types (Concept Maps) and create inline shimming functions, and to calculate the set of archetype types for a Concept. C++0x Concepts were difficult to inline and relied upon the optimizer to eliminate the overhead imposed by these automatically generated constructs. C++0x concepts also provided "definition checking", which was desirable in that a template function could be checked for correctness before instantiation. C++0x concepts used the "Concept Map" and archetype computations to decide whether a template would be correct. This compiletime computation of an archetype runs into a manifestation of the halting problem, which makes generalizing this solution to user defined concepts a problematic proposition at best.

By contrast, our solution does not involve the generation of such machinery. We require no generation of any adaptors, maps, or proxies. Instead, we propose altering and refining the lookup rules to further obey the restrictions imposed by Concepts in a manner similar to what is already in the existing design. We feel that this is appropriate, because Concepts already requires some alteration to the lookup rules, and our design appears to be consistent with the general lookup rule restrictions thereby imposed. Concept restrictions, in their current form, are enforced in C++ through lookup rules, not through any other mechanism. Our solution merely adds more rules to the set of lookup rules employed in concept processing. This also means that our solution is not capable of providing definition checking -- the lookup rules that this solution alters are those which occur during the second phase of name lookup, after template instantiation.

Q: Will I be able to call internal helper functions to my constrained function using an unqualified name?

A: Yes. We place no restrictions on the calling of functions in namespaces that are unrelated to the concept used in a constraint. The namespaces associated with the types that are constrained are also still searched, but only the names which are necessary (in some fashion) to meet the requirements of the concept are considered to be viable. In some sense this is the existing Concepts restriction on calling a constrained function applied in reverse -- constraints restrict which functions are called based upon their arguments. The current restriction prevents calling a function which is not prepared to accept a type. Our refinement prevents calling a function which is not presented as part of the requirements on a type. This example is illustrated in our "at scale" example, and it is one of our primary motivations.

Q: Isn't your real problem with {ADL, const vs. non-const overloads, overload resolution, dependent lookup, etc.} and not with the lookup rules of Concepts today?

A: Absolutely not. We have examples of unexpected selection of operation each and every one of these cases. We are not convinced that our problem is with every single one of the above aspects of the language. There are some cases

which will be redundantly resolved by improving those aspects of the language; however, many problem cases within each of these domains still remain. This is especially true of ADL functions. ADL functions are intended to be part of the interface of a class; however, a constrained value is also a constrained interface.

Q: How do I actually invoke ADL functions that I want invoked in my constrained functions?

A: ADL functions on an object which are actually part of the interface defined by the concept that constrains the calling function are still preserved as part of the viable overload set. This makes them likely to be the best match for an unqualified call, unless a more specific overload is available in some other reachable namespace. If you wish to call ADL functions which are not part of the constraint, then one always has two viable options. The first is to use a direct "using `std::foobar`" approach, and the second is to adjust the definition of a Concept to include this ADL operation.

Q: What about calling efficient swap on an Assignable?

A: This is actually a special case of the above concern. In this case, there are at least two viable options. The first is to add `swap( a, b )` to the requirements of the Assignable concept. The second is to make `std::swap` have an overload which accepts a value which is a model of the Swapable concept. An unqualified call to `swap` after the traditional using `std::swap;` declaration will invoke that Swapable overload, thus giving the correct behavior. In addition, this has the added benefit of making any direct call to `std::swap` in any context always take the best overload! Although this library change is not proposed by this paper, the authors would strongly support such a change.

Q: Is this going to give me better error messages?

A: Although this is highly dependent upon the details of an implementation, it is possible that better error messages would be possible under this proposal. The instantiation of a template which requires some specific operation which is not part of a concept should give a better error message -- something along the lines of "Function not found during constraint checking."

Q: What about implicit conversions needed in ?

A: Because the function selected by overload resolution must be part of the operations necessary to satisfy the constraints of the specified concept, all implicit conversions which are necessary to invoke that function are also part of the operations necessary to satisfy that concept. This means that any set of implicit conversions provided by a type which are necessary to invoke a selected overload should be "whitelisted" for use in constrained functions when calling that overload.

Q: Will I be able to invoke arbitrary operations on my constrained parameters which are not part of the concept?

A: Any function which is available directly in the namespace (directly or via a using statement) of a constrained function may be called. Any function which is a template but is not constrained will be called as an unconstrained function. In that context of an unconstrained template, any function may be invoked as normal. We also propose that an intrinsic cast-like operation could be added which will revert a constrained variable to an unconstrained variable, to permit calling functions in an unconstrained fashion for various purposes.

Design Considerations

Any design that proposes to change lookup rules should not invalidate code written under those rules today. Because of this, we do not propose that the lookup rules should be changed when evaluating names within a "classical" template

context. However, we see a number of opportunities to apply our modified lookup scheme:

Terse syntax constrained functions

The terse syntax intends to open generic programming to a wider audience, as discussed earlier. We feel that it is obvious that such terse syntax functions be subject to rules which provide a more intuitive result. Therefore we suggest that any terse syntax considered by the committee must have the intuitive semantics provided by our proposal.

Template syntax constrained function

All constrained function templates

Any expansion of a terse syntax from the terse form into a "canonical" production of a constrained template function declaration could automatically have these rules applied. This seems fairly obvious in many respects, because the purpose of Concepts is to afford better selection of applicable functions in name lookup and overload resolution. When a user writes a constrained function, even using template syntax, he or she is explicitly choosing to have the semantics of Concepts applied to their function. Therefore, it seems a reasonable choice to make every constrained function obey these lookup rules.

Opt-in for these rules as part of the definition of a constrained function template

A user trying to modernize a code base by adding constraints to existing template functions, may wind up causing subtle changes in the semantics and or ODR violations. Additionally, the template expert is already intimately familiar with the consequences of C++'s unintuitive lookup rules in templates and may wish to leverage the semantics afforded by these rules in the implementation of his template function -- he only wishes to constrain the callers, but not himself.

Therefore it may be necessary to control the application of this modified rule through the use of a signifying keyword. We propose `explicit template< ... >` as this syntax, as it reads reasonably well and clearly indicates intent. Although this proposed syntax uses the `template` keyword, which is already indicative of potential lookup dangers, it eschews the pitfalls for the `template< ... >` case.

Regardless of whether the new lookup rules are opt-in or opt-out, the language loses no expressivity. It is possible to choose the opposite alternative through other syntax.

Behavior of These Rules Under Short Circuit of Disjunction

We preserve the viability of overloads which are found on either branch of a disjunction, because a user would reasonably expect these overloads to be available if those constraints are satisfied. For branches of a disjunction which are not satisfied, those overloads will be unavailable, as the constraint wasn't satisfied. This seems to result in a viable overload set which most closely conforms to user expectations. The overall compile-time cost of this added checking should be proportional to the overall cost of treating a disjunction as a conjunction for this feature.

Conclusion

In P0726R0 it was asked if Concepts improve upon the expressivity of C++17. The response from EWG was mixed. The fact is that, although Concepts are more approachable and readable than C++17 `std::enable_if` predicates, they do not provide any new expressivity to the language, nor do they provide any facility for making templates actually easier to write. We feel that this proposal's adoption will give a new dimension to the Concepts feature which will enable the simple expression of C++ generic code in a much safer and more readable style than C++17 or the current Concepts design.

Revision History

P0782R0 - A Case for Simplifying/Improving Natural Syntax Concepts (Erich Keane, A.D.A. Martin, Allan Deutsch)

The original draft explained the general problem with a simple example. The motivation was to help attain a better consensus for "Natural Syntax" Concepts. It was presented in the Monday Evening session of EWG at Jacksonville, on 2018-03-12. The guidance from the group was strongly positive:

SF: 10 - F: 21 - N: 22 - A: 7 - SA: 1

Acknowledgements

The authors would like to thank Allan Deutsch, Hal Finkel, Lisa Lippincott, Gabriel Dos Reis, Sean Spillane, Herb Sutter, Faisal Vali, and numerous others for their research, support, input, review, and guidance throughout the lifetime of this proposal. Without their assistance this would not have been possible.

References

P0726R0 - "Does Concepts Improve on C++17?"